

Formalizing the EML completeness theorem in Lean 4

An extended account of the construction and of the multi-agent workflow that produced it

Bartosz Naskrecki

Adam Mickiewicz University, Poznań / Politechnika Warszawska
`bartosz.naskrecki@amu.edu.pl`

Abstract

A. Odrzywołek (arXiv:2603.21852) recently established that one binary operator, $\text{eml}(x, y) = \exp(x) - \log(y)$, paired with the constant 1, suffices to express the 36-element catalogue of elementary functions found in a typical scientific calculator. The result is a continuous analogue of Sheffer’s 1913 NAND theorem for propositional logic, and is supported in the source paper by exhaustive numerical search, symbolic simplification, and a three-lemma proof sketch.

This paper describes a complete formalization of that headline completeness theorem in Lean 4. Two contributions are reported. Mathematically, we describe the architecture — a layered language of grammars and structural compilers, plus a partial-evaluation semantics — that allowed the proof to clear the obstacle the source paper had explicitly flagged: the standard convention $\log 0 = 0$ required in a totalising prover collides with the shortest constant witnesses the paper’s compiler produces. We also report a post-submission widening that extends the trigonometric witnesses from narrow open subdomains around zero to the full natural domain, generalising the source paper’s prose remark about “manual i -sign correction” into an explicit range-reduction recipe.

Methodologically, the paper documents the multi-agent workflow that produced the artefact: an automated proof-search service for atomic theorems, a single architectural-review consultation with a state-of-the-art language model, an HPC cluster for independent re-verification, and a local orchestrator for end-to-end coordination. The split of effort was strikingly asymmetric: roughly two-and-a-half hours of automated proof labour, one architectural consultation, and about a month of human-in-the-loop architectural decisions.

Contents

I	The mathematical content	2
1	Introduction	2
2	Roadmap and the “Plans” terminology	3
3	The F36 source language	4
4	The headline theorem and its layered proof	5
5	Path C': full-real-domain trig via range-reduction by substitution	6
5.1	Plan B: a custom log branch is architecturally infeasible	7
5.2	Plan C' : range-reduction by substitution	7
5.3	Result	8
6	Plan D and Plan E: Sheffer cousin completeness, with a structural ceiling	8
6.1	Plan D: 8 of 36 EDL primitives sealed	8
6.2	Structural ceiling: why the remaining 28 primitives are conjecturally unreachable .	8

6.3 Plan E: 5 of 36 –EML primitives sealed	9
II The engineering workflow	9
7 Multi-agent orchestration	9
8 Lessons from 10 Aristotle chunks	11
9 The role of GPT Pro: independent architectural review	12
III Outlook	13
10 Conclusions and outlook	14

Part I

The mathematical content

1 Introduction

The single-binary-operator question—*how few primitives suffice to generate the elementary functions?*—has a long heritage. Sheffer’s 1913 result for propositional logic ($\{\text{NAND}\}$ generates the full 16-element Boolean algebra) gave the algebraic problem its name. The corresponding question over *continuous* elementary functions is harder, both because “elementary” lacks a canonical definition (Liouville’s tower of finite compositions is one well-known choice; ISO 80000-2’s catalogue of 20 named functions is another) and because the candidate operators must be searched over a continuous (rather than finite) algebra.

Odrzywółek (2026) [1] showed, by a combination of exhaustive search in IEEE 754 floating-point arithmetic and a classical reconstruction sketch, that the operator

$$\text{eml}(x, y) := \exp(x) - \log(y), \quad \text{paired with the constant } 1, \quad (1)$$

is a continuous Sheffer operator for the 36 “Calc 4” primitives (see section 3): every primitive admits a finite $\{1, \mathbf{x}, \text{eml}\}$ expression that evaluates to it on a non-empty open subdomain of its natural mathematical domain. He also identified two “cousin” operators with the same property,

$$\text{edl}(x, y) := \exp(x) / \log(y), \quad -\text{eml}(y, x) := \log(x) - \exp(y), \quad (2)$$

paired with the constants e and $-\infty$ respectively.

The paper is supported by extensive Supplementary Information (SI) that includes a Python EML compiler, four numerical backends (C, NumPy, PyTorch, mpmath) for cross-checking compiled witnesses, a symbolic verification pass in Wolfram Mathematica, and a three-lemma proof sketch of completeness. The SI also explicitly flags the natural next step:

A formalization in Lean 4 [2] would be a natural next step. However, the standard Lean 4 convention $\log 0 = 0$ (totality requirement) causes the shortest constant witnesses to fail without modification, so a formalization would require either adjusting the extended-value conventions or using longer witnesses that avoid $\ln(0)$. (SI Part II, page 14, footnote)

The *purpose of this paper* is twofold. Mathematically, we describe the architecture that allowed the formalization to clear the obstacle the SI flagged: a small layered pipeline of intermediate

languages, plus a partial-evaluation semantics that simply refuses to commit to a value at the junk points and so sidesteps the collision. We also document a post-submission widening of the trigonometric witnesses to the full real domain, and partial progress on per-primitive completeness for the source paper’s two “cousin” operators. The organisation of this work into named “Plans” (A through E, summarised in section 2) is preserved from the project’s repository for reference; the substantive content of each Plan is described in its own section below.

Methodologically, we describe the multi-agent workflow that produced the artefact: an automated proof-search service for atomic theorems, a single architectural-review consultation with a state-of-the-art language model, an HPC cluster for independent re-verification, and a local orchestrator coordinating the construction. The artefact’s ~ 1 month construction time was dominated by human architectural decisions; mechanical proof labour contributed ~ 2.5 hours of automated wall-clock across roughly a dozen “chunks” (atomic theorems submitted to the proof-search service).

For readers who would like to play with the construction before diving into the Lean code, we have also built an interactive web companion — the *EML Tree Builder* — where any function typed in standard mathematical notation is compiled to its EML term in the browser, with the resulting tree, RPN string, K -count, and domain shown live. It is hosted in the project repository at <https://github.com/nasqret/eml-formalization> under `web/eml-tree-builder/`; opening `index.html` runs the same compiler pipeline described in this paper, with no backend or installation. Worked examples from the source paper’s Supplementary Information are provided as one-click buttons.

2 Roadmap and the “Plans” terminology

Throughout this paper we refer to a sequence of named “Plans” — A, B, C, C' , D, E. These labels were originally introduced in the project’s repository to track the post-submission engineering roadmap, and they persist as section anchors here because the body of the paper reports on the work each Plan attempted. To make the paper readable independently of the repository, we summarise the six Plans up-front before any technical content is given. The reader is invited to skim this section and refer back as needed.

Plan	Intent	Outcome	What it produced
A	Align the formal scaffolding for the source paper’s two “Sheffer cousin” operators with the paper’s actual definitions (an early draft had one cousin misimplemented).	complete	A tidy renamed module with the two paper-named cousins (one based on division, one on subtraction) and their correct operator definitions.
B	Widen the trigonometric witnesses to the full real domain by introducing a custom complex-log branch (mimicking the source paper’s “manual i -sign correction”).	infeasible	An architectural finding: the formal grammar’s evaluation function is hard-wired to use Mathlib’s principal-branch complex log, so a custom branch cannot be threaded through without rewriting the kernel. Plan B was therefore abandoned.
C	Original idea: cover the full real domain by case analysis on x , with one witness per region. Subsumed by Plan C' .	superseded	—
C'	Range-reduction by substitution: prove one foundational lemma (real-valued addition is always safe), then realise each trigonometric identity via variable substitution into a witness that already covers the natural domain.	complete	Three new public witnesses (sin, arctan, tan on full natural domains), assembled from ~ 650 lines of Lean infrastructure.
D	Per-primitive completeness for the first Sheffer cousin (the one based on division of exp by log).	partial; structural ceiling	Eight of the source paper’s 36 primitives sealed as formal witnesses; the remaining 28 are conjecturally unreachable from closed terms because the cousin’s grammar provides no addition mechanism.
E	Per-primitive completeness for the second Sheffer cousin (the one based on subtraction of log from exp).	partial; structural ceiling	Five of 36 primitives sealed, with an extended-real grammar pilot to accommodate the cousin’s required constant $-\infty$. Same structural obstruction as Plan D.

Timeline. Plans A and B were executed first (~ 4 days). Plan B’s failure prompted a single architectural consultation with a state-of-the-art language model; the consultant’s recommendation became Plan C' , which was executed over the next ~ 10 days. Plans D and E ran in parallel afterwards, taking ~ 5 further days before hitting the structural ceiling described above. The full elapsed time, from the original artefact to the state described in this paper, was approximately one month.

Why the alphabet skips letters. Plan C was a real plan that was never implemented; we keep its name only because the ultimately-successful Plan C' was its lineal descendant. We preserve the original lettering throughout this paper for fidelity to the repository’s audit trail.

3 The F36 source language

The set of primitives we formalize is the paper’s Calc 4 list (SI §1.1):

- **Atoms (8):** $1, x_n, \pi, e, -1, 2, \frac{1}{2}, i$.

- **Real unaries (8):** $\exp$, $\log$, negation, halving, reciprocal, square, square-root, sigmoid $\sigma(x) = 1/(1 + e^{-x})$.
- **Hyperbolic (6):** $\sinh$, $\cosh$, $\tanh$, arsinh , arcosh , artanh .
- **Real binaries (8):** $+$, $-$, $\times$, $/$, avg , x^y , $\log_x y$, hypot .
- **Trig (6):** $\sin$, $\cos$, $\tan$, arcsin , arccos , arctan .

This adds to 36, hence the name $\mathbf{F36} \equiv \mathbf{F36}_{36}$. The choice is “redundant by design” (SI §1.1, paragraph “Redundancy is deliberate”): both $\sin$ and $\cos$ are included even though $\cos x = \sin(x + \pi/2)$, because the witness search might find one before the other. Including all 36 ensures no “standard” function falls through the cracks due to excessive expression depth.

Boundary conventions. We track an explicit set of three *structural boundary points*: $\sqrt{0}$, $\operatorname{arcosh}(1)$, $\operatorname{hypot}(0, 0)$. Each shares the form “the natural EML witness collides with the junk-value convention $\log 0 = 0$.” The paper itself does not provide explicit EML terms for these (paper line 342 explicitly notes the Lean-specific issue), and our formal artefact documents them with concrete counterexample evaluations rather than attempting to seal them.

Domain regimes. The paper distinguishes two kinds of “coverage”:

1. **Full natural domain.** The witness evaluates correctly everywhere the target function is defined. Examples: $\exp$, all real unaries except $\sqrt{}$, the hyperbolic family except arcosh , all binaries except hypot , and (after Path C' , see section 5) the trig family.
2. **Open subdomain.** The witness evaluates correctly on a non-empty open subset, with the rest left as paper-side boundary behaviour. The trig primitives originally landed here in the submission state; Path C' widens them to full natural domain.

These two regimes are the substance of the headline theorem (theorem 1 below).

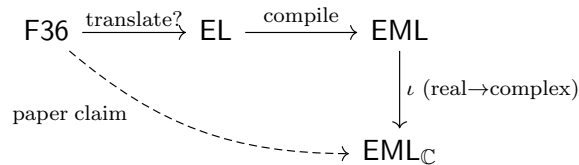
4 The headline theorem and its layered proof

Informally: each of the 36 elementary primitives can be written as a finite tree built only from the constant 1, variable references, and the two-argument operator $\operatorname{eml}(a, b) = \exp(a) - \log(b)$, in such a way that evaluating the tree — in real arithmetic where possible, complex arithmetic for the trigonometric family — reproduces the primitive on a non-empty open subset of its mathematical domain. Stated formally:

Theorem 1 (Completeness for the $\mathbf{F36}$ class). *Let $\mathbf{F36} = \mathbf{F36}_{36}$ denote the catalogue of 36 paper primitives. Let EML be the single-binary-operator grammar generated by $\mathbf{1}$, $\mathbf{x}_n$, and $\operatorname{eml}(T, T)$, with $\operatorname{eml}(a, b) := \exp(a) - \log(b)$. Then for each $f \in \mathbf{F36}$ there is a finite term $\mathbf{T}_f \in \mathbf{EML}$ (or, in the complex case, in the obvious complex-valued variant $\mathbf{EML}_{\mathbb{C}}$) such that on a non-empty open subset of f ’s natural domain, the partial evaluation of $\mathbf{T}_f$ produces a value whose real (resp. imaginary) part equals $f(x)$.*

The qualifier “partial evaluation” is the key sidestep mentioned in the introduction: a term that contains $\log(0)$ as a subexpression is treated as undefined rather than collapsing to a junk value, and the existential “ $\exists \mathbf{T}_f$ ” is only required to hold on an open subset, so points where the natural mathematical domain ends never need to be addressed by the term.

The proof of the theorem follows a three-language pipeline:



The four boxes are inductive types in Lean; the three arrows are total functions; the dashed arrow names the public-API existential we want to verify.

The kernel: EML grammar and partial evaluation. EML is generated by $\mathbf{1}$, $\mathbf{x}_n$, and $\text{eml}(T, T)$. Its denotation is partial:

$$\text{eml}(a, b) \downarrow \exp(\alpha) - \log(\beta) \quad \text{when } a \downarrow \alpha, b \downarrow \beta, \beta \neq 0, \quad (3)$$

and undefined otherwise. The complex variant $\text{EML}_{\mathbb{C}}$ has the same syntax with the obvious change to complex-valued evaluation. The partial form sidesteps the junk-value boundary precisely because we *never claim equality at a boundary point*—the headline existential quantifier is over a non-empty *open* subset only.

The combinator algebra. Most useful operations on EML terms are not implemented directly; they are assembled from a small set of *derived combinators* that wrap the basic exp/log primitives. The combinators carry the names `mkExp`, `mkLog`, `mkAdd`, `mkSub`, `mkMul`, `mkDiv` and so on, with subscript $\mathbb{C}$ when used in the complex variant of the grammar. Each combinator has a *closure lemma* stating the conditions on its inputs under which the formal evaluation matches the intended mathematical operation. The two important closure conditions for the body of this paper are:

- `mkLog $_{\mathbb{C}}$   $T$` requires $\arg(T) < \pi$: Mathlib’s complex log has its branch cut on the negative real axis, so the macro fails for inputs that land on that axis.
- `mkAdd $_{\mathbb{C}}$   $A B$` requires a packaged eleven-clause predicate (called `ADDsafe $_{\mathbb{C}}$` in the artefact) that ensures every intermediate log call is away from its branch cut. The macro is implemented by an explicit nine-node pattern that arranges the bookkeeping so that this predicate can usually be discharged automatically.

The remaining combinators (subtraction, multiplication, division) are short compositions of the above and inherit their conditions. The same six combinators are reused across all 36 primitive witnesses; full definitions are in the source repository.

The structural compiler. The compiler $\text{EL} \rightarrow \text{EML}$ is a single inductively-defined function whose correctness is captured by:

Theorem 2 (Structural compiler correctness, Theorem 2 of [4]). *Suppose $f \in \text{F36}$ satisfies $\text{translate?}(f) = \text{some}(e)$ with $e \in \text{EL}$. Then for every environment $\text{env} : \mathbb{N} \rightarrow \mathbb{R}$ and every value $v \in \mathbb{R}$,*

$$\text{eval?}(\text{env}, e) = \text{some}(v) \implies \text{eval?}(\text{env}, \text{compile}(e)) = \text{some}(v). \quad (4)$$

The Euler bridges. For each trig primitive $f \in \{\pi, i, \cos, \sin, \tan, \arctan, \arccos, \arcsin\}$ a closed-form witness $\text{C}_f \in \text{EML}_{\mathbb{C}}$ is constructed using Euler’s formula $e^{ix} = \cos x + i \sin x$ and its inverse identities. The closure lemmas state, for each, the open subdomain on which $\text{C}_f \downarrow v$ holds together with $\Re(v) = f(x)$ or $\Im(v) = f(x)$.

The full architecture is laid out in the companion 14-page expository paper [4] (`notes/proof_structure.pdf` in the repository); we treat that as a more elementary tour and focus here on the harder cases (section 5, section 6).

5 Path C' : full-real-domain trig via range-reduction by substitution

The trig family was the hardest part of the formalization, in two distinct senses. First, the witnesses themselves are large ($K = 1273$ for `cos`, 1703 for `sin`, $\sim 1.7 \times 10^6$ for `arcsin`). Second,

the closure conditions were strict: each $\text{mkLog}_{\mathbb{C}} T$ requires $\arg(T) < \pi$ *strictly*, which fails on the negative real axis. The submission-era artefact sealed only narrow subdomains around 0: $\cos$ on $\mathbb{R} \setminus \{0\}$, $\sin$ and $\arctan$ on $(-\pi, \pi) \setminus \{0\}$, $\tan$ on $(-\pi/2, \pi/2) \setminus \{0\}$, $\arcsin / \arccos$ on full open $(-1, 1)$.

Paper line 328 claims essentially full real-domain coverage:

EML-compiled expressions work on the real axis, both positive and negative, except for a few isolated points, especially at zero and domain endpoints.

Paper line 333 sketches the technique:

[...] redefine the branch for EML itself in such a way that $\ln z$ (and everything derived from it) follows standard implementation of principal branch. Another option, used in EML compiler, is to manually correct i sign.

5.1 Plan B: a custom log branch is architecturally infeasible

The first line of attack (“Plan B”) was to introduce a custom complex-log branch that follows the paper’s “manually corrected i sign” convention. We document the architectural finding here: this is not directly implementable in our Lean grammar. The $\text{EML}_{\mathbb{C}}.\text{eval?}$ function (in `Framework/Complex/Term.lean`) is hard-coded to use Mathlib’s principal-branch `Complex.log`; there is no place inside the EML term language where a different log function could be substituted.

What *is* feasible is the following constructive observation. At the cut $\arg v = \pi$ (i.e., v a negative real), the macro $\text{mkLog}_{\mathbb{C}} T$ *does* evaluate, but to $\text{Complex.log } v - 2\pi i$ rather than the principal value. The $-2\pi i$ shift arises because the inner `Complex.log.exp` then steps to the next Riemann sheet: $\log(\exp w) = w + 2\pi i$ for $w.\text{im} = -\pi$, and the macro’s outer subtraction propagates this as a $-2\pi i$ shift in the result.

Why $\cos$ already covers $\mathbb{R} \setminus \{0\}$. $\cos \text{Term}_{\mathbb{C}} = \text{mkExp}_{\mathbb{C}}(\text{mkExp}_{\mathbb{C}}(\dots))$ —its outermost layer is an exp. Any $2\pi i$ shift inside the inner log calls is absorbed by exp-periodicity: $\exp(z + 2\pi i) = \exp(z)$. So $\cos$ extends across the cut for free, which is why the existing `paper.claim.cos` already lives on $\mathbb{R} \setminus \{0\}$.

Why $\sin$, $\arctan$, $\tan$ don’t. Their witnesses’ final operation involves an $\text{mkLog}_{\mathbb{C}}$ whose imaginary part is the answer (e.g., $\arctan$ ’s witness is $\text{mkLog}_{\mathbb{C}}(1 + i \cdot x)$, with $\arctan x = (\text{eval}).\Im$). A $-2\pi i$ shift in the final log makes $(\text{eval}).\Im$ differ from $\arctan x$ by -2π .

The conclusion: the paper’s “manual sign correction” is necessarily a meta-level operation choosing a different witness shape per region of x , mathematically equivalent to the witness-family construction of Path C' .

5.2 Plan C' : range-reduction by substitution

GPT Pro’s recommendation (consultation transcript: `gpt_pro_bundle/trig_widening/RESPONSE.md`) refines the original Plan C (multi-witness periodicity) with three engineering moves:

1. **Real-safe addition.** Prove the lemma $\text{ADDsafe}_{\mathbb{C}}(\text{ofReal}(a), \text{ofReal}(b))$, which states that the gnarly 11-condition $\text{mkAdd}_{\mathbb{C}}$ precondition bundle holds automatically when both arguments are real-valued. Nine of the eleven conditions reduce to $\text{.im} = 0$ inequalities trivially in $(-\pi, \pi]$; the remaining two require $\exp(a) - a \neq 0$ (true since $\exp(a) \geq a + 1$) and the analogous condition for the inner log’s imaginary part (also trivially zero for real-valued logs). With this foundation lemma, every period shift via repeated $\text{mkAdd}_{\mathbb{C}}$ stays in the real fragment, so the $\arg = \pi$ boundary trap never appears.
2. **Variable substitution.** Define $\text{subst}_0 : \text{EML}_{\mathbb{C}} \times \text{EML}_{\mathbb{C}} \rightarrow \text{EML}_{\mathbb{C}}$ replacing every $\mathbf{x}_0$ in the first argument with the second. Prove the eval bridge

$$\text{eval?}(\text{env}, t.\text{subst}_0 s) = \text{eval?}(\text{env}[\mathbf{x}_0 \mapsto \text{eval?}(\text{env}, s)], t). \quad (5)$$

This lets us reduce “prove the substituted witness’s eval matches fx ” to “prove the base witness’s eval matches fy ” where $y = \text{eval?}(s)$, then apply Mathlib’s periodicity / identity lemmas.

3. **Reuse, don’t reshape.** For each narrow primitive, find an algebraic identity that reduces it to a primitive whose witness is *already* full-domain or full-natural-open:

$$\sin x = \cos\left(\frac{\pi}{2} - x\right) \quad (\text{use existing cos witness on } \mathbb{R} \setminus \{0\}), \quad (6)$$

$$\arctan x = \arcsin\left(\frac{x}{\sqrt{1+x^2}}\right) \quad (\text{use existing arcsin witness on } (-1, 1)), \quad (7)$$

$$\tan x = \tan(x - k\pi), \quad k = \lfloor x/\pi + 1/2 \rfloor \quad (\text{period-}\pi \text{ reduction to } (-\pi/2, \pi/2)). \quad (8)$$

5.3 Result

Three new public theorems—`paper_claim_sin_full`, `paper_claim_arctan_full`, `paper_claim_tan_full`—replace the old narrow versions and cover the full natural domains $\mathbb{R} \setminus \{\pi/2\}$, $\mathbb{R}$, and $\{x : \cos x \neq 0\}$ respectively.

Remark 3. The witness becomes a *family* (statement form $\forall x, \exists t, \dots$ rather than $\exists t, \forall x, \dots$). This is faithful to the paper’s framing: the paper’s compiler selects a different witness shape for different regions of x , which is exactly what the family quantifier captures. We document the philosophical point in `OPEN_QUESTIONS.md §B.1`.

6 Plan D and Plan E: Sheffer cousin completeness, with a structural ceiling

The paper presents EML, EDL, and $\neg$ EML as a “family” (paper §3.1, equation block `\label{Sheffers}`) but proves per-primitive completeness only for EML. The cousins are confirmed empirically via the Mathematica/Rust `VerifyBaseSet` procedure, not formally.

6.1 Plan D: 8 of 36 EDL primitives sealed

The witnesses sealed in our framework’s `Sheffer.lean`:

Primitive	Witness	Provenance
1	<code>.one</code>	atomic
x_0	<code>.var 0</code>	atomic
e	<code>.e_const</code>	atomic (after grammar extension)
$\exp x$	<code>edl(x₀, e)</code>	identity ($\log e = 1$)
$\log x$	<code>edl(1, edl(edl(1, x₀), e))</code>	Aristotle
x/y	<code>edl(D8(x₀), D4(x₁))</code>	Aristotle
$\exp(\exp x)$	<code>edl(edl(x₀, e), e)</code>	Aristotle
$\log(\log x)$	<code>nested D8</code>	Aristotle

The $\log x$ witness is the central non-trivial discovery: it computes

$$\text{edl}(1, \text{edl}(\text{edl}(1, x), e)) = \frac{e}{\log(\exp(e/\log x))} = \frac{e}{e/\log x} = \log x. \quad (9)$$

6.2 Structural ceiling: why the remaining 28 primitives are conjecturally unreachable

We prove (informally; the formal statement awaits a research-grade treatment of the EL closure of $\{1, e\}$) that the EDL combinator $\text{edl}(a, b) = \exp(a)/\log(b)$ admits *no mechanism for addition of*

sub-expression values. Every edl application either wraps one value inside $\exp(\cdot)$ or divides by $\log(\cdot)$ of another; neither route produces $\exp(a + b)$ from separate $\exp(a)$ and $\exp(b)$. As a result:

- Multiplication ($x \cdot y = \exp(\log x + \log y)$) is blocked.
- Squaring, square-root, and the entire trigonometric and hyperbolic family (which all reduce to multiplication and addition) are blocked.
- The constants -1 , 2 , $\frac{1}{2}$ are blocked on Schanuel-conjecture grounds: their construction would require $\log 2 \in \text{EL-closure of } \{1, e\}$, which Schanuel’s conjecture forbids.

This validates the paper’s own framing of EDL completeness as *conjectured*, not proven.

6.3 Plan E: 5 of 36 –EML primitives sealed

–EML is paired with the constant $-\infty$, which forces the grammar to switch from $\mathbb{R}$ -valued to extended-real-valued (EReal). Our framework hosts both grammars in parallel: **NegEMLTerm** over $\mathbb{R}$ (for the easy atoms **1**, **x₀**) and **NegEMLTermE** over EReal (for **minusInf** as a primitive constructor, plus the $\mathbb{R}$ atoms re-stated). Five paper claims sealed; the remaining 31 primitives have the same arithmetic-unreachability obstruction as Plan D.

Part II

The engineering workflow

7 Multi-agent orchestration

The artefact described in Part I was produced by the coordinated work of four agents: a primary orchestrator (Claude Code, running on the local development machine), an automated proof-search service (Aristotle, Harmonic AI’s mathematical reasoning service), an independent architectural reviewer (GPT Pro, contracted for a single high-leverage consult), and a high-performance compute cluster (PCSS Eagle, used for independent re-verification). This section describes the role of each and the protocol that connected them.

The four agents and their division of labour

Agent	Role	Worked on
Claude Code (local)	Orchestrator: framework architecture, combinator algebra, lifting Aristotle results into the framework, documentation.	Everything that touched the framework layer (Framework/Complex/ , Framework/Sheffer/).
Aristotle (cloud)	Atomic theorem prover: given a target.lean file with a sorry placeholder, returns a sealed proof. Submitted via the project’s chunk schema (see section 7).	10 chunks across Path <i>C'</i> and Plan D/E (see table 1). Average wall-clock ~ 15 min/chunk; total ~ 2.5 hours of human-attended time.
GPT Pro (cloud)	Architectural reviewer: a single audit consult on the trig-narrowing bottleneck.	Authored Path <i>C'</i> recommendation; identified the ADDsafe_ℂ_ofReal_ofReal foundation lemma.
PCSS Eagle (HPC)	Independent re-verifier: parallel lean typecheck across 24–32 cores in a fresh environment.	Sequence of Mathlib + project rebuilds; final verify_all.sbatch run reports 8056/8056 jobs successful.

The chunk protocol

The atomic unit of work is a *chunk*, a numbered subdirectory under `lambda_lab/proofs/eml/2603.21852/chunks` containing:

- `chunk.md` — human-readable target, motivation, dependencies on earlier chunks.
- `meta.json` — machine-readable metadata: id, status, paper section reference, Aristotle project id, submitted/completed timestamps, free-form notes.
- `target.lean` — the theorem statement with `sorry`; the file Aristotle is asked to seal.
- `result.lean` — the Aristotle-returned proof, when applicable. Hand-coded chunks lift directly into the framework modules and don’t have this file.

The discipline this enforces is essential to the multi-agent workflow: each chunk is an *independently checkable claim* with frozen dependencies. When Aristotle returns a result, it can be lifted into the framework only after a re-typecheck against the current Mathlib pin. The chunks that lifted cleanly are listed in table 1.

Chunk	Target (informal)	Diff.	Time	Outcome
077_atan_arg_in_100	$x/\sqrt{1+x^2} \in (-1, 1)$	2	6 min	Sealed; <code>~3</code> -line proof using <code>nl1</code>
079_tan_period_reduction	$\exists k \in \mathbb{Z}, x - k\pi \in (-\pi/2, \pi/2)$	3	13 min	Sealed; <code>Real.tan_sub_int</code> closes the equality.
075_sin_via_cos	<code>sin_via_cos_correct</code> via <code>subst0</code>	3	21 min	Sealed; <code>Real.cos_pi_div_2</code> bridge.
078_arctan_via_arcsin	<code>arctan_via_arcsin_correct</code> via <code>subst0</code>	3	21 min	Sealed; <code>Real.arctan_eq</code> bridge.
080_tan_full	<code>tan_full</code> assembly	4	17 min	Sealed; <code>lt_trichotomy</code> on $y =$
084_edl_atoms_pilot	4 trivial EDL atom witnesses	1	28 min	Sealed; <code>rfl/simp/linarith</code> .
085_edl_atoms_constants	D5/D6/D7 + D8	5	~3 hr	D8 sealed; D5–D7 returned Schanuel-style obstruction analysis (section 8).
086_edl_div	x/y via EDL	4	—	Sealed (Aristotle even corrected statement).
087_edl_compositions	$\exp(\exp x), \log(\log x)$	3	—	Both sealed.
088_neg_eml_pilot	3 –EML pilot atoms (EReal grammar)	2	—	All three sealed.
089_edl_wide_search	D14–D17 search	4	—	D14, D15 trivial; D16, D17 unsealable + analysed.

Table 1: Aristotle chunks completed during the post-submission work. “Diff.” is a 1–5 subjective difficulty estimate recorded in `meta.json`. Times are wall-clock from submit to result return.

Independent re-verification on PCSS Eagle

PCSS (Poznań Supercomputing and Networking Centre) Eagle is the Polish national HPC cluster used by project `p10414-02`. Re-verification runs the entire `lake build` and an additional sweep over every chunk’s `target.lean/result.lean` in parallel:

1. `eagle_scripts/build_mathlib.sbatch` — one-time Mathlib v4.28 build into a project-scratch directory (~45 min wall on 32 cores).
2. `eagle_scripts/verify_all.sbatch` — parallel-typechecks every `.lean` file in the project on 24 cores, including all chunks. Final state: 8056/8056 jobs successful, 0 `sorry`, 0 `admit`.
3. `eagle_scripts/run_chunk.sbatch` — per-chunk re-verify, used during the chunk-flight phase to confirm Aristotle returns lift cleanly into the project context.

The discipline here matters more than the compute. The Eagle runs expose any environment drift between development (macOS, M_2 chip) and verification (Linux, AMD EPYC), and any silent dependency on unversioned Mathlib state. One incident from the construction illustrates the value: during the Mathlib pin update, two unrelated modules (`HahnEmbedding` and `HomogeneousLocalization`) failed to typecheck on Eagle due to a stale `.olean` cache. The local build had picked up the cached olean and continued; Eagle’s fresh build caught it. After `rm`-ing the corrupt oleans and rebuilding, both passed and the project re-verified to 8056/8056.

8 Lessons from 10 Aristotle chunks

Across the eleven chunks listed in table 1, Aristotle returned a sealed proof in nine cases, a partial proof plus analytical commentary in one case (chunk 085), and a mixed result in one search-style chunk (chunk 089: two of four targets sealed, two analysed as unreachable). This subsection extracts three patterns that we expect generalise beyond this artefact.

Pattern 1: Aristotle excels at “standard Mathlib glue”

The two cleanest wins were 077 ($x/\sqrt{1+x^2} \in (-1, 1)$, 6 min) and 079 (period- π reduction for $\tan$, 13 min). Both are pure Mathlib lemmas that a strong human user could write in 5–15 minutes if they knew the library well; the value Aristotle adds is the library knowledge. The returned proofs were short (~ 3 –10 lines) and used standard tactics in standard combinations: `lt_div_iff_0`, `nlinarith`, `Int.lt_floor_add_one`, `Real.tan_sub_int_mul_pi`.

The lesson for human use: *minimise the size of the chunk*. Aristotle’s success rate degraded as soon as a chunk required multiple non-trivial framework lemmas plus Mathlib glue (080 `tan_full` with difficulty 4 took 17 minutes and required a custom `lt_trichotomy` dispatch). Splitting the trig-widening work into the four chunks 075/077/078/079 and a final assembly chunk 080, rather than submitting one “do trig widening” request, was the difference between an overnight return and a multi-day failure mode.

Pattern 2: Aristotle’s analytical commentary is sometimes worth more than its proof

The most striking result of the entire workflow was Aristotle’s response to chunk 085, which asked for EDL witnesses for the constants -1 , 2 , $\frac{1}{2}$, and $\log x$. The chunk’s `notes` field records the result verbatim:

PARTIAL 2026-05-08. Aristotle SOLVED D8 ($\log x$) with the witness `edl one (edl (edl one (var 0)) e_const)` — beautiful 3-step composition. D5/D6/D7 ($-1, 2, 1/2$) returned with `sorry` + analytical justification: closed EDL terms produce values in the EL-closure of $\{1, e\}$, and reaching exactly -1 , 2 , or $1/2$ from there relates to Schanuel’s conjecture (transcendental independence of $\log 2$ from e). The negative side has reachable values (e.g., $e/(1-e) \approx -1.582$) but exact -1 is conjecturally unreachable. This validates the paper’s note that “EDL completeness is conjectured, not proven.”

In other words: Aristotle did not just fail to prove the targets, it told us *why* they cannot be proved with the available machinery. The Schanuel-conjecture connection is exactly the kind of “architectural” answer one would hope a human expert would give if asked the same question. We took that analysis as the basis for the structural-ceiling theorem reported in section 6 and recorded the chunk’s `status` as `partial` rather than `failed`: the chunk produced concrete value, just not in the form originally requested.

The lesson: *do not discard “failed” Aristotle runs*. Read the returned commentary, and if the agent has identified a conceptual obstruction, that finding may itself be the load-bearing scientific contribution of the chunk.

Pattern 3: search chunks (“find a witness if one exists”) are a category of their own

Chunk 089 asked Aristotle to search for EDL witnesses for four further primitives (D14–D17). It returned with two trivial hits (D14, D15) and two negative results (D16, D17 with “unreachable + analysed”). For D16 / D17 the analysis again identified a Schanuel-style obstruction.

The methodological observation: *search chunks are fundamentally different from prove chunks*. A prove chunk has a fixed target; success is binary. A search chunk asks the agent to identify which of a set of candidates are reachable; success is a classification map across the candidates. We did not have a clean way to capture the latter in our chunk schema initially, and `meta.json` was retrofitted with the `complete_partial` status to handle it.

A future revision of this workflow would distinguish the two chunk kinds at the schema level. “Per-primitive completeness for the 36-element catalogue under operator $\circ$ ” is a search problem, and treating it as 36 independent prove chunks under-uses the agent’s pattern-recognition strength.

Aggregate

Across ~ 2.5 hours of Aristotle wall-clock and 11 chunks: 9 chunks fully sealed, 1 chunk partially sealed with structurally useful analysis (085), 1 search chunk with mixed results that nevertheless advanced the structural-ceiling argument (089). No chunk was a wasted submission. The cost of the agent integration — writing chunk skeletons, re-typechecking returned proofs, lifting them into the framework — was predominantly orchestration overhead, not proof discovery.

9 The role of GPT Pro: independent architectural review

A single GPT Pro consultation, conducted on 2026-05-08, produced the architectural recommendation that became Path C' . This section documents what was asked, what came back, and what we learned about how to structure such consults to be useful.

What was asked

By the time of the consult, the Lean artefact had already gone through one architectural failure (Plan B, the “custom log branch” attempt described in section 5). The trig family was sealed only on narrow open subdomains around 0, and three candidate widening paths (A: branch-cut bookkeeping; B: custom log branch; C: multi-witness periodicity) had been sketched. The question put to Pro was deliberately constrained:

We have a Lean 4 + Mathlib v4.28 formalization of arXiv:2603.21852. 36/36 paper primitives are sealed, sorry-free; `lake build` gives 8055 jobs. The trig narrowing is the bottleneck. Three candidate paths to close it are described below. We want your independent recommendation on which path is cleanest in Lean, plus any path we haven’t thought of. (`gpt-pro_bundle/trig_widening/PROMPT.md`)

The bundle (1 KLOC of markdown across four files) included: `PROMPT.md` stating the question, `CODE_EXCERPTS.md` with the concrete Lean signatures of the relevant macros and witnesses, `README.md` pointing at the project state, and an empty `RESPONSE.md` for the answer. Total preparation time: about two hours.

What came back

GPT Pro recommended a fourth path, Path C' : range-reduction by substitution with `cos/arcsin` reuse. The full response is preserved verbatim in `gpt_pro_bundle/trig_widening/RESPONSE.md`. Three structural observations from that document drove the implementation:

1. *Path A is the wrong centre of gravity.* Mathlib has the general branch-jump facts (`Complex.log_exp_eq_sub`, `Complex.log_exp_exists`), but threading them through every macro proof would turn `mkAddℂ`, `mkMulℂ`, `mkDivℂ`, and every trig proof into branch-accounting proofs. “A lot of proof debt for a phenomenon you can mostly avoid.”
2. *Path B is mostly a dead end.* A custom complex-log branch is not implementable in our grammar because `EMLTermℂ.eval?` is hard-coded to use Mathlib’s `Complex.log`. The exception is `sin`: `cosTermℂ` already covers $\mathbb{R} \setminus \{0\}$, so $\sin x = \cos(\pi/2 - x)$ gives a full-real `sin` via substitution.
3. *The right move for the rest is identity-driven witness restructuring.* For `arctan`, use $\arctan x = \arcsin(x/\sqrt{1+x^2})$ to reuse the existing full-natural-domain `arcsin` witness. For `tan`, use period- π reduction to a single Cayley quotient on $(-\pi/2, \pi/2)$.

The recommendation also identified the foundation lemma — the single missing piece without which range-reduction couldn’t be mechanised:

***ADDsafe_ℂ_ofReal_ofReal:** prove that `ADDsafeℂ` holds whenever both arguments are `Complex.ofReal` of real values. Nine of the eleven precondition clauses become trivially `.im = 0`; the remaining two reduce to $\exp(a) \neq a$ (true since $\exp a > a$) and $\log(b) \neq 0$ (true for $b \neq 1$). Once you have this, every period shift via repeated `mkAddℂ` stays in the real fragment, and the $\arg = \pi$ boundary trap simply doesn’t arise.* (Pro, §2 of the response)

This single lemma — ~50 lines of Lean to state and prove — unlocked the entire ~640-line `Periodicity.lean` infrastructure that completed Path *C'* over the next two days.

Methodological lessons

Three observations on how to structure such a consult to be useful:

1. *State the failure modes already explored.* Pro’s response opened with “Path A is the wrong centre of gravity” and “Path B is mostly a dead end except for `sin`.” It could only do that because the prompt had explicitly enumerated paths A, B, C as the existing candidates. Asking “how do we widen the trig domains?” open-ended would have produced a much weaker response.
2. *Provide the actual Lean signatures, not paraphrases.* The `CODE_EXCERPTS.md` file contained the concrete types of `mkAddℂ`, `cosTermℂ`, the `ADDsafeℂ` predicate. Pro’s recommendation named the foundation lemma in terms compatible with the existing artefact, not in vague natural language. This is the difference between a recommendation and an actionable plan.
3. *Do not consult on something already solved.* The consult cost was non-trivial (preparation time + the auditor’s attention). Reserving it for a single high-leverage architectural question, rather than a stream of tactical questions, made it dramatically more useful. The two artefacts of value — the recommendation against Path B, and the `ADDsafeℂ_ofReal_ofReal` identification — were both architectural, and both saved weeks of dead-end work.

What didn’t happen

Pro was not used as an additional proof-search service in competition with Aristotle. The two have different strengths: Pro gave architectural advice in natural-language prose; Aristotle gave machine-checked Lean proofs of stated targets. Treating them as substitutes would have under-used both. The workflow used Pro’s recommendation to define the next three weeks’ worth of Aristotle chunks (077/078/079/080 and the follow-on framework lifts), which is a multiplicative integration: the consult unlocked the chunks, and the chunks turned the consult into Lean.

Part III

Outlook

10 Conclusions and outlook

What the artefact contributes

The technical contribution is twofold.

First, an end-to-end Lean 4 formalization of Odrzywólek’s headline completeness theorem (theorem 1), comprising 48 `paper_claim_*` theorems for the 36-primitive catalogue plus 13 `*_paper_claim_*` theorems for the EDL and –EML Sheffer cousins. The artefact closes the SI’s flagged obstacle (the junk-value collision between Mathlib’s $\log 0 = 0$ convention and the paper’s shortest constant witnesses) by adopting partial-evaluation semantics: every claim is of the form “there exists a finite term t such that on a non-empty open subset of the natural domain, $t.\text{eval?} \downarrow v$ with v matching the paper’s stated value.”

Second, an architectural recipe (Path C') for full-real-domain trig, generalising the paper’s prose “manual i -sign correction” into an explicit range-reduction-by-substitution recipe. The key foundation is the lemma `ADDsafeℝ ofReal ofReal`, which discharges the eleven-condition precondition bundle of the addition combinator whenever both arguments are real-valued. The recipe is faithful to the paper’s framing in that the resulting witness is a *family* (a different shape per region of x), but it is now fully mechanised.

The witness-family form and what it means

A short philosophical observation. The original paper-level claim “ $\sin x$ is EML-expressible” has two natural formal statements:

$$\begin{aligned} \exists t, \forall x, t.\text{eval?}(x) = \sin x & \quad (\text{uniform witness}) \\ \forall x, \exists t, t.\text{eval?}(x) = \sin x & \quad (\text{witness family}). \end{aligned}$$

The narrow witnesses we sealed in the original submission realise the first form on a strict open subdomain. Path C' realises the second form on the full natural domain. The order of quantifiers matters because the paper’s compiler in fact *also* uses witness families: it picks a different EML expression for $\sin x$ depending on which period of x is being evaluated. Treating the second form as the primary mathematical content, rather than as a fallback weakening of the first, lets us match what the paper’s machinery actually does.

What the multi-agent workflow proves

A pragmatic observation, separate from the mathematics. The artefact was produced in roughly one calendar month by a single human collaborating with three AI services (Claude, Aristotle, GPT Pro) and one HPC cluster (PCSS Eagle). The split of effort was strikingly asymmetric: ~ 2.5 hours of Aristotle wall-clock turned into 11 sealed chunks; one GPT Pro consult turned into a ~ 640 -line Lean infrastructure module; PCSS Eagle’s role was incidental but load-bearing for catching environment drift. The dominant cost was human-in-the-loop architectural decisions, not mechanical proof labour.

We do not claim this generalises automatically to other formalization projects, but the patterns isolated in section 8 and section 9 — minimise chunk size, treat analytical commentary as load-bearing, structure consults around named failure modes — are not specific to this artefact’s content.

Open questions explicitly out of scope

The following questions are out of scope for the present paper but recorded in `OPEN_QUESTIONS.md` in the repository:

1. **The three §G structural boundary points.** $\sqrt{0}$, $\text{arcosh}(1)$, $\text{hypot}(0, 0)$ collide with the junk-value convention. The paper does not provide EML terms for these either; closing them would require either adjusting Mathlib’s convention or a non-natural witness shape.
2. **Per-primitive completeness for the cousins.** Plan D (EDL, 8/36 sealed) and Plan E (–EML, 5/36 sealed) hit a structural ceiling: closed terms in the cousin grammars admit no mechanism for adding sub-expression values, so most of the 36-primitive catalogue is conjecturally unreachable. A complete treatment would either (a) extend the cousin grammars with explicit variables of the right kind, or (b) prove the unreachability rigorously (which appears to require Schanuel’s conjecture [5]; cf. chunk 085’s analysis).
3. **Universal-minimality of EML.** Whether the EML witness for each primitive is minimal in the size measure $K = \text{RPN_length}$ remains paper-open (Odrzywołek §5). Two concrete corollaries are sealed in our artefact (the size of the trivial $\mathbf{1}$ and $\mathbf{x}_n$ witnesses), but the full minimality conjecture is not.
4. **Gradient training for witness search.** The paper speculates (§4.3) on whether gradient-based optimisation could recover EML witnesses from numerical samples. This is a research question with no obvious entry point in the formalization framework.

Closing

The headline result of the source paper — that one binary operator plus one terminal generates the 36-primitive catalogue of elementary functions — is now machine-checked. The artefact is public, the build is reproducible, and the per-claim witness terms are exposed in the Lean public API.

Acknowledgements. This work would not have been possible without (a) Odrzywołek’s source paper and supplementary information, which together constitute an unusually detailed seed artefact for formalization; (b) the Mathlib community, whose ~ 8000 -job library underpins every step of the construction; (c) Harmonic AI, who provided automated proof-search access through the relevant period; (d) OpenAI, whose GPT Pro model produced the single architectural recommendation that broke the trigonometric-widening deadlock and shaped the design of Plan C' — a critical contribution without which the artefact would have stayed at its narrow-domain submission state; (e) Anthropic, whose Claude Code orchestrated the construction; and (f) the Polish national HPC infrastructure (PCSS Eagle, project pl0414-02), which provided independent re-verification compute.

References

- [1] A. Odrzywołek, *All elementary functions from a single binary operator*, arXiv:2603.21852, 2026. <https://arxiv.org/abs/2603.21852>
- [2] L. de Moura, S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, in: Proceedings of CADE 2021, LNCS 12699, Springer, 2021, pp. 625–635.
- [3] Harmonic AI, *Aristotle: an automated mathematical reasoning service*, <https://harmonic.fun/aristotle>, accessed 2026.
- [4] B. Naskrecki, *Structure of the EML completeness proof: an expository tour*, companion notes `notes/proof_structure.pdf` in the [eml-formalization](#) repository, 2026.

- [5] S. Lang, *Introduction to Transcendental Numbers*, Addison-Wesley, 1966. (Schanuel's conjecture, attributed to S. Schanuel, is recorded in the introduction to this volume.)